

Number of Longest Increasing Subsequences

Comprehensive Technical Guide • Exact DP State Transitions • Complete Interview Dry Run

Core Problem Distinction

The problem "Number of Longest Increasing Subsequences" (LeetCode 673) is fundamentally different from the standard LIS length problem.

Important Point: Patience Sorting ($O(n \log n)$) cannot be directly modified to count the number of LIS. The standard interview solution requires Dynamic Programming in $O(n^2)$.

1. Intuition & State Representation

Previously, for finding the standard LIS length, we stored:

dp[i] = Length of the LIS ending at index *i*.

Now, this is not enough. We also need to know: *How many LIS of this maximum length end at index i?*

So we maintain a second parallel array:

count[i] = Number of LIS ending at index *i*.

DP State Declaration

```
vector<int> dp(n, 1);  
vector<int> count(n, 1);
```

- **dp[i]**: Length of the LIS ending at *i*.
- **count[i]**: Number of LIS ending at *i*.

Base Initial State

Every element alone forms a valid LIS of length 1.

Example element: 5

LIS length = 1 | Number of LIS = 1

Therefore: **dp[i] = 1; count[i] = 1;**

2. DP State Transitions

For every pair of indices $j < i$, if $nums[j] < nums[i]$, then $nums[i]$ can extend the LIS ending at *j*. There are two distinct cases:

Case 1: Found a Strictly Longer LIS

Condition: $dp[j] + 1 > dp[i]$

Example: Current $dp[i] = 2$. We found $dp[j] + 1 = 4$. Obviously, length 2 is no longer useful. Replace it!

```
dp[i] = dp[j] + 1;  
count[i] = count[j];
```

*Notice: We copy the count because all best LIS ending at *j* now become best LIS ending at *i*.*

Case 2: Found Another Way to Obtain Same Length

Condition: $dp[j] + 1 == dp[i]$

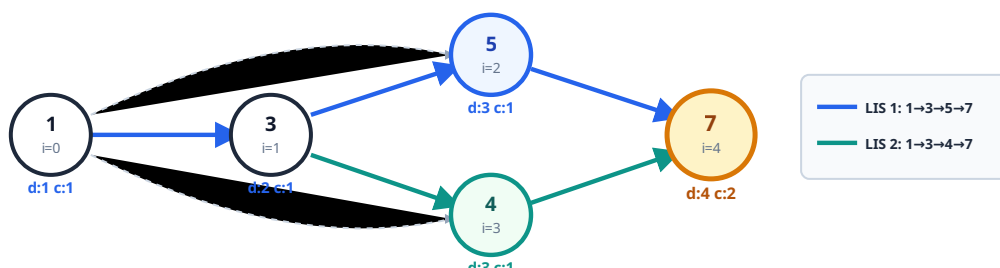
Example: Length 4 is already possible. Now another previous element also gives Length 4. Additional LIS discovered!

```
count[i] += count[j];
```

*Notice: We accumulate counts because we discovered an additional optimal path ending at *i*.*

3. Subsequence Branching & Decision Visualizations

GRAPH DIAGRAM: SUBSEQUENCE TRANSITION PATHS FOR $nums = [1, 3, 5, 4, 7]$



SUBTREE DIAGRAM: EVALUATION OF ELEMENT $nums[4] = 7$

nums[4] = 7 (Target)

4. Algorithm & High-Level Execution

```
// Dynamic Programming Algorithm Structure
Initialize dp[i] = 1, count[i] = 1 for all i from 0 to n-1

For every i from 0 to n-1:
    For every j from 0 to i-1:
        If nums[j] < nums[i]:
            If dp[j] + 1 > dp[i]:
                dp[i] = dp[j] + 1
                count[i] = count[j]
            Else if dp[j] + 1 == dp[i]:
                count[i] += count[j]
```

Dry Run High-Level Progression: nums = [1, 3, 5, 4, 7]

Index ()	Value (nums[i])	dp[i] (Max LIS Length)	count[i] (Ways to reach)	State Notes
0	1	1	1	Base element
1	3	2	1	Extended from j=0
2	5	3	1	Extended from j=1
3	4	3	1	Extended from j=1
4	7	4	2	Case 1 from j=2 (dp=4, c=1) + Case 2 from j=3 (c += 1 → 2)

Final High-Level Summary

Maximum LIS Length (maxi): 4

Number of LIS (ans): 2

The Two Unique LIS Paths

- 1 → 3 → 5 → 7
- 1 → 3 → 4 → 7

5. Complete C++ Implementation

```
class Solution {
public:
    int findNumberOfLIS(vector<int>& nums) {
        int n = nums.size();

        vector<int> dp(n, 1);
        vector<int> count(n, 1);

        int maxi = 1;

        for(int i = 0; i < n; i++) {
            for(int j = 0; j < i; j++) {
                if(nums[j] < nums[i]) {

                    // Found a longer LIS
                    if(dp[j] + 1 > dp[i]) {
                        dp[i] = dp[j] + 1;
                        count[i] = count[j];
                    }

                    // Found another LIS of the same maximum length
                    else if(dp[j] + 1 == dp[i]) {
                        count[i] += count[j];
                    }
                }
            }
            maxi = max(maxi, dp[i]);
        }

        int ans = 0;

        for(int i = 0; i < n; i++) {
            if(dp[i] == maxi)
                ans += count[i];
        }

        return ans;
    }
};
```

```
}  
};
```

6. Complexity Analysis & Key Takeaways

Time Complexity

Outer loop: $O(n)$ iterations.

Inner loop: $O(n)$ comparisons per index.

Overall Time: $O(n^2)$

Space Complexity

dp array: $O(n)$ space for lengths.

count array: $O(n)$ space for counts.

Overall Space: $O(n)$

Key Points to Remember

- $dp[i]$ = length of the LIS ending at index i .
- $count[i]$ = number of LIS ending at index i .
- If a longer LIS is found ($dp[j] + 1 > dp[i]$), update length and **copy** the count:
 $dp[i] = dp[j] + 1$; $count[i] = count[j]$;
- If another LIS of the same maximum length is found ($dp[j] + 1 == dp[i]$), **add** the counts:
 $count[i] += count[j]$;
- After filling the DP arrays, find the maximum LIS length ($maxi$) and sum the counts of all indices having that length. This gives the total number of longest increasing subsequences.

7. Complete Interview-Style Dry Run

Let's do a complete interview-style dry run of the DP + Count algorithm for: `nums = [1, 3, 5, 4, 7]`

Step 1: Initialization

SETUP

We create two arrays:

$dp[i]$ = Length of LIS ending at i | $count[i]$ = Number of LIS ending at i

Initially, every element alone is an LIS.

Index	0	1	2	3	4
Value	1	3	5	4	7
dp	1	1	1	1	1
count	1	1	1	1	1

Maximum LIS length so far: $maxi = 1$

$i = 0$ • Current element `nums[0] = 1`

$i = 0$

There is no previous element ($j < 0$). Nothing changes.

dp: [1, 1, 1, 1, 1] | count: [1, 1, 1, 1, 1] | maxi: 1

$i = 1$ • Current element `nums[1] = 3`

$i = 1$

We check every previous element $j < 1$:

```
j = 0 ( nums[0] = 1 ): 1 < 3 ✓  
Possible new length = dp[0] + 1 = 1 + 1 = 2 . Current dp[1] = 1 .  
Since 2 > 1, we found a longer LIS!  
Update: dp[1] = 2 , count[1] = count[0] = 1 .
```

Arrays become: dp: [1, 2, 1, 1, 1] | count: [1, 1, 1, 1, 1] | Update maxi = 2

$i = 2$ • Current element `nums[2] = 5`

$i = 2$

We check every previous element $j < 2$:

```
j = 0 ( nums[0] = 1 ): 1 < 5 ✓  
Possible length = 1 + 1 = 2 . Current dp[2] = 1 .  
Since 2 > 1, longer! Update: dp[2] = 2 , count[2] = count[0] = 1 .  
Arrays: dp: [1, 2, 2, 1, 1], count: [1, 1, 1, 1, 1]
```

$j = 1$ ($\text{nums}[1] = 3$): $3 < 5$ ✓
 Possible length = $\text{dp}[1] + 1 = 2 + 1 = 3$. Current $\text{dp}[2] = 2$.
 Since $3 > 2$, longer! Update: $\text{dp}[2] = 3$, $\text{count}[2] = \text{count}[1] = 1$.

Arrays become: dp : [1, 2, 3, 1, 1] | count : [1, 1, 1, 1, 1] | Update $\text{maxi} = 3$

$i = 3$ • Current element $\text{nums}[3] = 4$

$i = 3$

We check every previous element $j < 3$:

$j = 0$ ($\text{nums}[0] = 1$): $1 < 4$ ✓ → Possible length = 2. Current $\text{dp}[3] = 1$.
 Update: $\text{dp}[3] = 2$, $\text{count}[3] = 1$.
 Arrays: dp : [1, 2, 3, 2, 1], count : [1, 1, 1, 1, 1]

$j = 1$ ($\text{nums}[1] = 3$): $3 < 4$ ✓ → Possible length = $\text{dp}[1] + 1 = 2 + 1 = 3$. Current $\text{dp}[3] = 2$.
 Since $3 > 2$, longer! Update: $\text{dp}[3] = 3$, $\text{count}[3] = \text{count}[1] = 1$.
 Arrays: dp : [1, 2, 3, 3, 1], count : [1, 1, 1, 1, 1]

$j = 2$ ($\text{nums}[2] = 5$): $5 < 4$ ✗ (Cannot extend. Nothing changes).

Final after $i = 3$: dp : [1, 2, 3, 3, 1] | count : [1, 1, 1, 1, 1]

$i = 4$ • Current element $\text{nums}[4] = 7$ (Pivotal Step)

$i = 4$ FINAL

We check every previous element $j < 4$:

$j = 0$ ($\text{nums}[0] = 1$): $1 < 7$ ✓ → Possible length = 2. Update: $\text{dp}[4] = 2$, $\text{count}[4] = 1$.
 Arrays: dp : [1, 2, 3, 3, 2], count : [1, 1, 1, 1, 1]

$j = 1$ ($\text{nums}[1] = 3$): $3 < 7$ ✓ → Possible length = 3. Update: $\text{dp}[4] = 3$, $\text{count}[4] = 1$.
 Arrays: dp : [1, 2, 3, 3, 3], count : [1, 1, 1, 1, 1]

$j = 2$ ($\text{nums}[2] = 5$): $5 < 7$ ✓
 Possible length = $\text{dp}[2] + 1 = 3 + 1 = 4$. Current $\text{dp}[4] = 3$.
 Since $4 > 3$, longer! Update: $\text{dp}[4] = 4$, $\text{count}[4] = \text{count}[2] = 1$.
 Arrays: dp : [1, 2, 3, 3, 4], count : [1, 1, 1, 1, 1]

$j = 3$ ($\text{nums}[3] = 4$): $4 < 7$ ✓
 Possible length = $\text{dp}[3] + 1 = 3 + 1 = 4$. Current $\text{dp}[4] = 4$.
 Notice $4 == 4$! This means we found another way to obtain an LIS of length 4.
 The existing LIS was $1 \rightarrow 3 \rightarrow 5 \rightarrow 7$. Now we found another $1 \rightarrow 3 \rightarrow 4 \rightarrow 7$.
 So we do NOT replace. Instead, we add the counts:
 $\text{count}[4] += \text{count}[3] \rightarrow \text{count}[4] = 1 + 1 = 2$.

Arrays become: dp : [1, 2, 3, 3, 4] | count : [1, 1, 1, 1, 2] | Update $\text{maxi} = 4$

8. Final Arrays & Extracting the Answer

Index	0	1	2	3	4
Value	1	3	5	4	7
dp	1	2	3	3	4
count	1	1	1	1	2

Finding the Answer

Maximum LIS length: $\text{maxi} = 4$.
 Now sum the counts of all indices having $\text{dp}[i] == 4$:
 Only Index 4 has $\text{dp}[4] == 4$, with $\text{count}[4] = 2$.
Final Answer = 2

Why is the Answer 2?

There are exactly two longest increasing subsequences of length 4:

- LIS 1: $1 \rightarrow 3 \rightarrow 5 \rightarrow 7$
- LIS 2: $1 \rightarrow 3 \rightarrow 4 \rightarrow 7$

These two different paths both end at 7, so $\text{count}[4] = 2$.

The Two Update Rules to Memorize

When processing `nums[j] < nums[i]` :

Rule 1: Found a longer LIS (`dp[j] + 1 > dp[i]`)

Update both: `dp[i] = dp[j] + 1; count[i] = count[j];`

Why? Because you've found a strictly better (longer) subsequence, so you discard the old length and inherit the number of ways from j.

Rule 2: Found another LIS of the same length (`dp[j] + 1 == dp[i]`)

Update only the count: `count[i] += count[j];`

Why? Because you've discovered an additional way to build the same optimal length ending at i, so you add the number of ways instead of replacing them. This distinction is the core idea behind counting LIS.